

CSIP5102 (Data Warehouse) Assignment

Group 1:

Nisha Khadka BK

TASK 1

1.1 Discuss the business scenarios and analyse the given database design and the requirement for a data mart and identify the dimensions and fact for your data mart.

Review of the Analysis Requirement for the Business Scenario

- A. The total sale value of each production
- B. Monthly sale value of each theatre
- C. The theatre name (for each theatre) and the names of clients who have the highest spending in that theatre

The query is to analyse the sale value by production, theatre and client and the time granularity is Month.

Declaration of the Grain

- A. The client grain is a specific client
- B. The production grain is a specific production-level
- C. The theatre grain is a specific theatre
- D. The time grain is a specific month

1.2 Dimensions and Fact Table

Dimension Tables:

Based on the business scenario and the brief, only the essential dimensions were included:

- Dimension table for Client - Group1_dc
- Dimension table for Theatre - Group1_dt
- Dimension table for Production – Group1_dp
- Dimension table for Time (Month) – Group1_dm

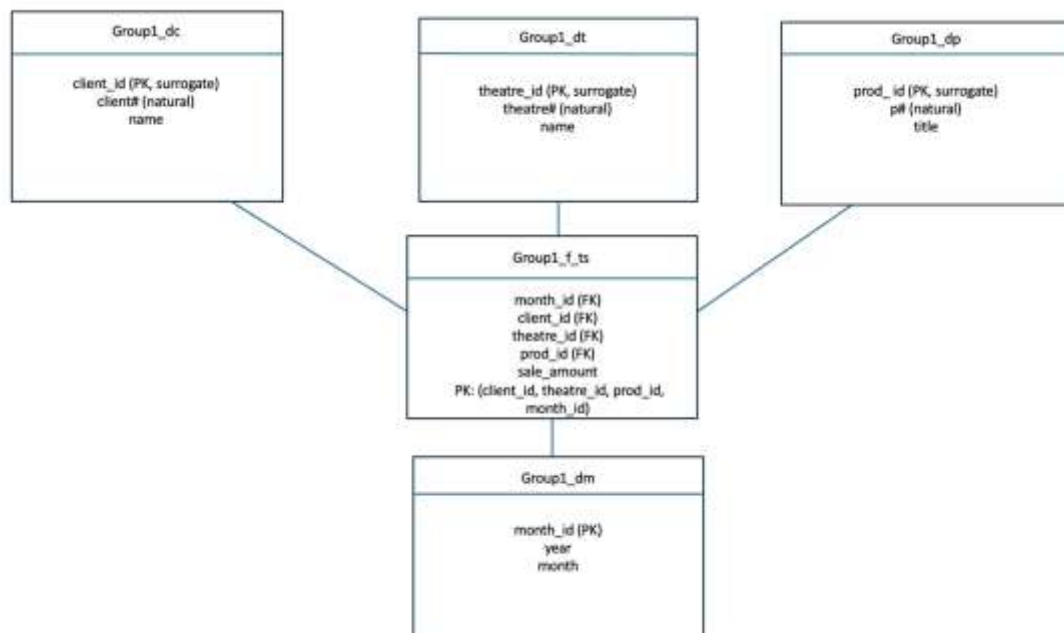
NOTE: We **excluded** Payment, Delivery, Performance and the Full Date dimensions because none of the analytical queries required it. The brief's "minimum essential dimensions only" criteria was followed.

Fact Table:

- Fact table for Ticket Sales - Group1_f_ts

TASK 2

2.1 Based on your group discussion in task 1, design the star schema for the data mart



Fact Table: Group1_f_ts

The fact table represents ticket sales business process, it is a quantitative measure and contains an additive measure which is the sale amount.

Dimension Tables: We have 4 dimension tables in the analysis.

- A. **Dimension 1: Production** (Group1_dp) - Essential for the total sale by production.
- B. **Dimension 2: Theatre** (Group1_dt) - Essential for the monthly sale by theatre, top clients per theatre.
- C. **Dimension 3: Client** (Group1_dc) - Essential for the client names with highest spending.
- D. **Dimension 4: Month** (Group1_dm) - Essential for the monthly sale value. Month is the granularity because requirement is monthly.

2.2 Determine the relevant attributes

- A. **Dimension Tables** – the below are the keys in the dimension tables

Natural key: Group1_sc uses client# and Group1_dp uses p# as their natural keys. Group1_dt uses theatre#.

Group1_dm is a monthly time dimension and its natural key is month#.

Surrogate key: client_id is the surrogate key for Group1_dc, prod_id for Group1_dp, theatre_id for Group1_dt and month_id for Group1_dm.

Primary key: client_id in Group1_dc; prod_id in Group1_dp; theatre_id in Group1_dt; month_id in Group1_dm.

- B. **Fact Table - the below are the keys in the fact table**

Primary keys: Group1_f_ts primary key is the composite of client_id, theatre_id, prod_id, and month_id.

Foreign keys: in Group1_f_ts are client_id referencing Group1_dc, theatre_id referencing Group1_dt, prod_id referencing Group1_dp, month_id referencing Group1_dm.

2.3 Declaration of the Grain

A. The client grain is a specific client

Justification: This is essential for the theatre name (for each theatre) and the names of clients who have the highest spending in that theatre.

B. The production grain is a specific production-level

Justification: This is essential for the total sale value of each production.

C. The theatre grain is a specific theatre

Justification: This is essential for the monthly sale value of each theatre and the top clients by theatre.

D. The time grain is a specific month

Justification: This is essential for the monthly sale value of each theatre. We selected month because the requirement is to analyse by "Month".

TASK 3

3.1 Map your star schema to logical relations

Logical relations and mapping:

A. Dimension Tables

Group1_dc (client_id, Client#, title, name, town)

Group1_dt (theatre_id, Theatre#, name, town, county)

Group1_dp (prod_id, P#, title, production_director, play_author)

Group1_dm (month_id, year, month)

B. Fact Table

Group1_f_ts (client_id, theatre_id, prod_id, month_id, sale_amount)

Mapping

The Group1_f_ts fact table is linked to the Group1_dc (client) dimension through client_id surrogate key.

The Group1_f_ts fact table is linked to the Group1_dp (production) dimension through prod_id surrogate key.

The Group1_f_ts fact table is linked to the Group1_dt (theatre) dimension through theatre_id surrogate key.

The Group1_f_ts fact table is linked to the Group1_dm (month) dimension through month_id surrogate key.

3.2 Identify the Primary Keys (PK) and Foreign Keys (FK)

A. Dimension Tables Primary Keys

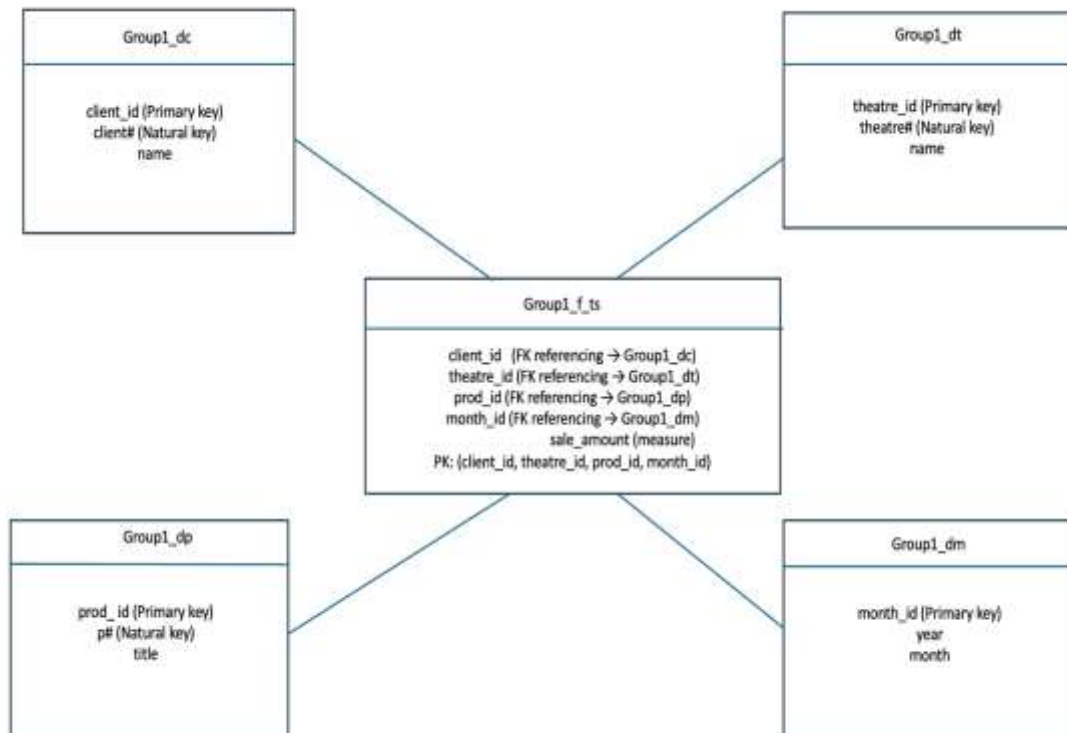
- Group1_dc: PK = client_id
- Group1_dt: PK = theatre_id
- Group1_dp: PK = prod_id
- Group1_dm: PK = month_id

B. Fact Table Primary Keys and Foreign Keys

Group1_f_ts: composite primary key = (client_id, theatre_id, prod_id, month_id)

Foreign keys in Group1_f_ts

- client_id FK for Group1_dc
- theatre_id FK for Group1_dt
- prod_id FK for Group1_dp
- month_id FK for Group1_dm



Task 4:

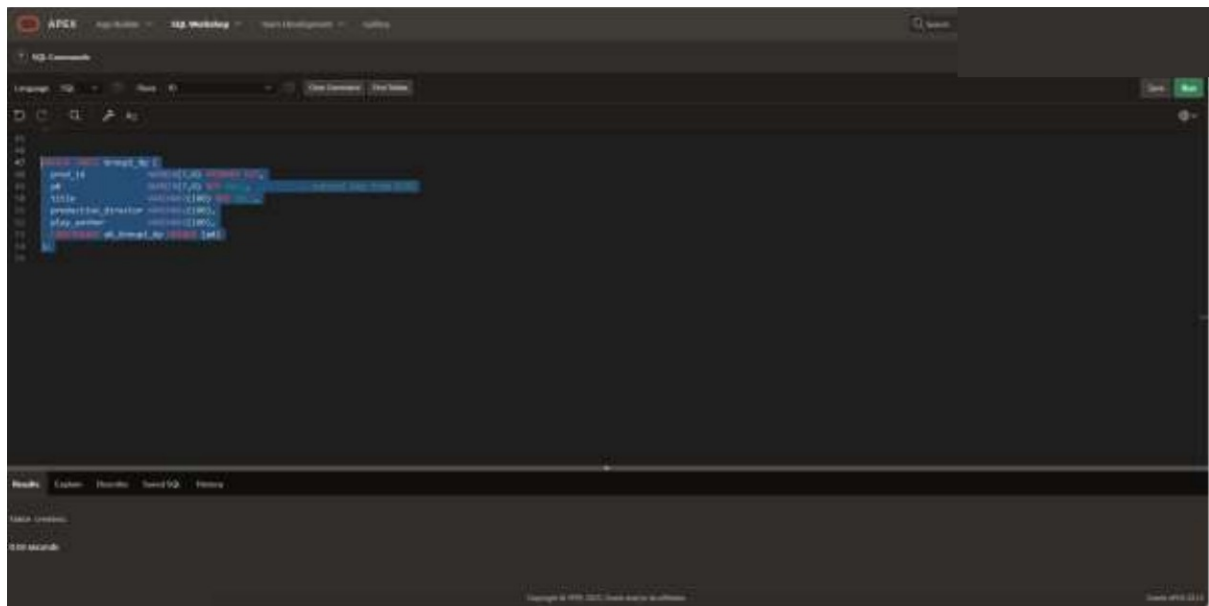
Create the corresponding tables in Oracle using SQL

Group1_dp (prod_id, P#, title, production_director, play_author)

-- Dimensions

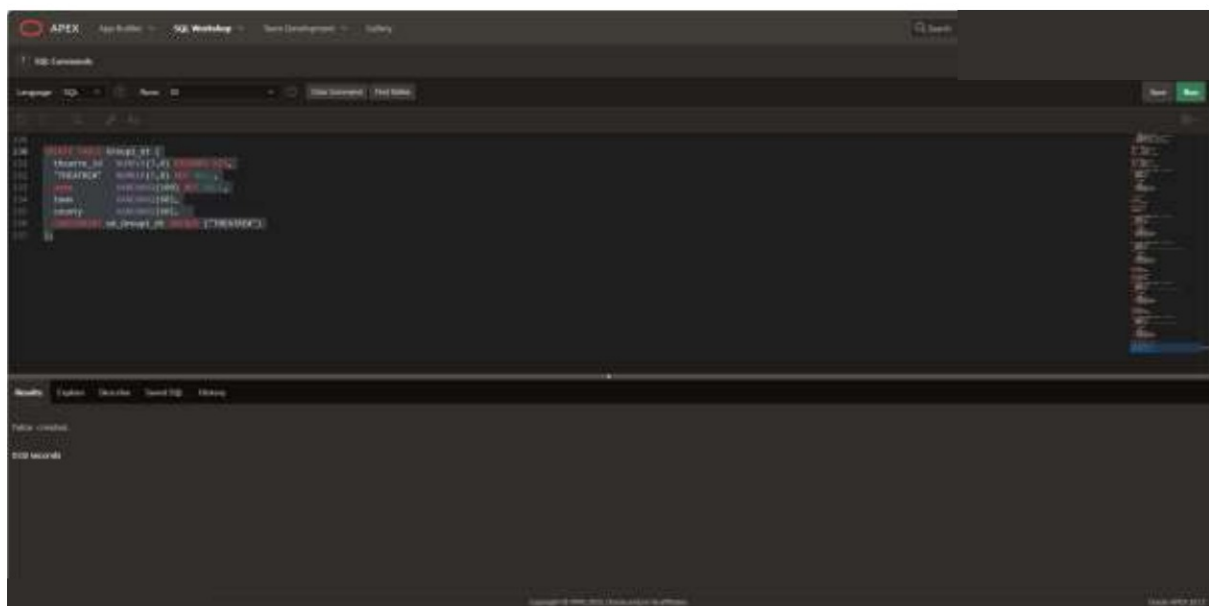
```

CREATE TABLE Group1_dp (
  prod_id      NUMBER(7,0) PRIMARY KEY,
  p#           NUMBER(7,0) NOT NULL,      -- natural key from OLTP
  title        VARCHAR2(100) NOT NULL,
  production_director VARCHAR2(100),
  play_author   VARCHAR2(100),
  CONSTRAINT uk_Group1_dp UNIQUE (p#)
);
  
```



Evidence of the code working and table created: FROM APEX
Group1_dt (theatre_id, Theatre#, name, town, county)

```
CREATE TABLE Group1_dt (
  theatre_id  NUMBER(7,0) PRIMARY KEY,
  "THEATRE#" NUMBER(7,0) NOT NULL,
  name       VARCHAR2(100) NOT NULL,
  town       VARCHAR2(60),
  county     VARCHAR2(60),
  CONSTRAINT uk_Group1_dt UNIQUE ("THEATRE#")
);
```

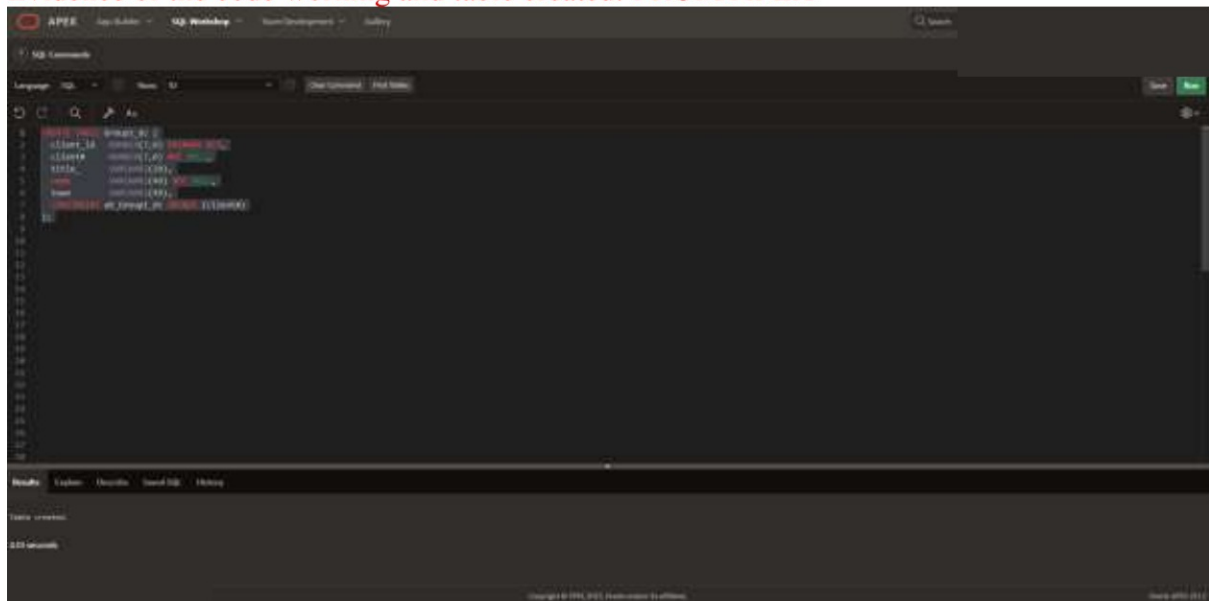


Evidence of the code working and table created: FROM APEX

Group1_dc (client_id, Client#, title, name, town)

```
CREATE TABLE Group1_dc (  
  client_id  NUMBER(7,0) PRIMARY KEY,  
  client#   NUMBER(7,0) NOT NULL,  
  title_    VARCHAR2(20),  
  name      VARCHAR2(40) NOT NULL,  
  town      VARCHAR2(40),  
  CONSTRAINT uk_Group1_dc UNIQUE (client#)  
);
```

Evidence of the code working and table created: FROM APEX



Group1_dm (month_id, year, month)

```
CREATE TABLE Group1_dm (  
  month_id  NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,  
  year      NUMBER(4) NOT NULL,  
  month     VARCHAR2(20) NOT NULL,  
  CONSTRAINT uk_Group1_dm UNIQUE (full_date)  
);
```

Evidence of the code working and table created: FROM APEX

-- FACT TABLE

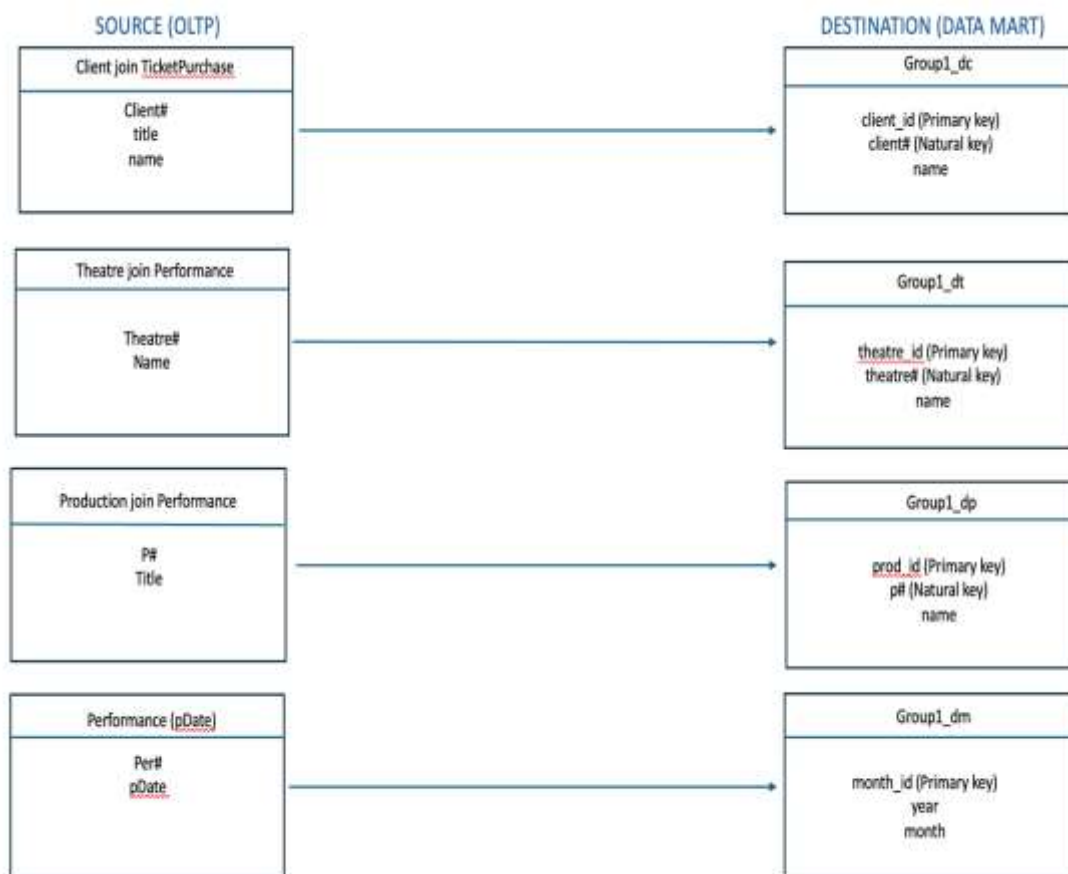
```
CREATE TABLE group1_f_sales (  
  prod_id   NUMBER NOT NULL  
    REFERENCES group1_dim_production(prod_id),  
  theatre_id NUMBER NOT NULL  
    REFERENCES group1_dim_theatre(theatre_id),  
  client_id NUMBER NOT NULL  
    REFERENCES group1_dim_client(client_id),  
  date_id   NUMBER NOT NULL  
    REFERENCES group1_dim_date(date_id),
```

amount NUMBER(12,2) NOT NULL,
 CONSTRAINT pk_group1_f_sales PRIMARY KEY (prod_id, theatre_id, client_id, date_id)
);

Evidence of the code working and table created: FROM APEX

Task 5:

5.1 Identify your source data from the OLTP database and design your data extraction rules.



5.2 Data Extraction Rules

Rule	Rule Enforcement	Rule Application
Sales-Only Constraint	Dimensions and fact	Loaded only rows that appear in TicketPurchase

Grain enforcement	Dimensions and fact	Fact is monthly at (client_id, theatre_id, prod_id, month_id); composite PK on these four keys in the Fact table.
Time grain	Dimension table - Group1_dm grain is in month	Derive month_id = YYYYMM from Performance.pDate; keep year
Preserve natural keys	All dimensions	OLTP IDs Natural Keys: Client#, Theatre#, P#
Generate surrogate PKs	All dimensions	Created surrogate keys are client_id, theatre_id, prod_id, month_id; used in joins
Referential Integrity	All foreign keys are referencing primary keys in the dimension table	FKs in Group1_f_ts client_id referencing Group1_dc theatre_id referencing Group1_dt prod_id referencing Group1_dp month_id referencing Group1_dm
Aggregation	Fact table	The sale_amount is aggregated as the sum of the TotalAmount at the monthly grain

TASK 6

Implement your data extraction, transformation and loading through Oracle SQL. The number of rows extracted into each dimension or fact table in your data mart should be printed from Oracle query.

Using the star schema name for OLTP access ops\$yyang00.* provided

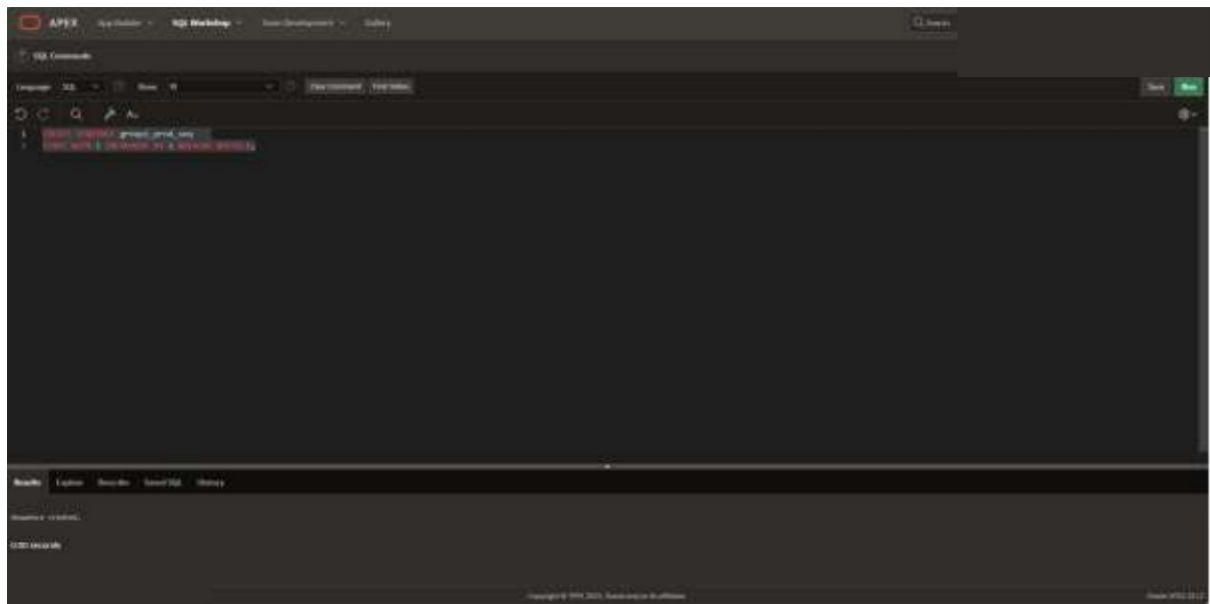
Below is the fully aligned version that *will run successfully* if your dimension and fact tables exist as defined in Task 4 (Group 1).

6.1 Create Sequences

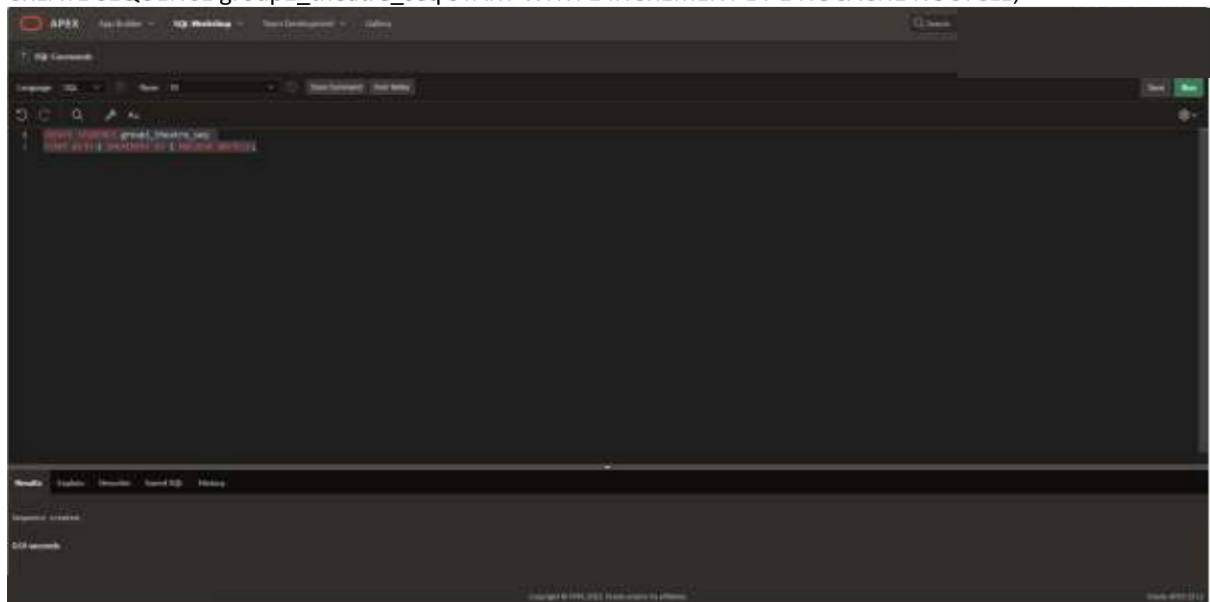
CREATE SEQUENCE group1_client_seq START WITH 1 INCREMENT BY 1 NOCACHE NOCYCLE;



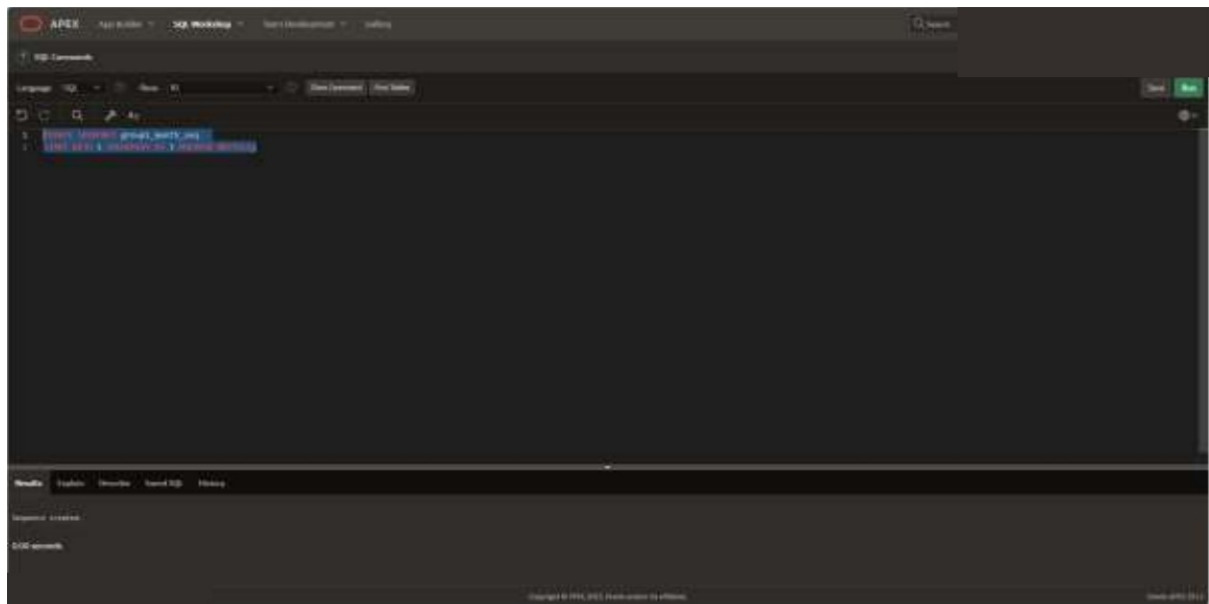
CREATE SEQUENCE group1_prod_seq START WITH 1 INCREMENT BY 1 NOCACHE NOCYCLE;



CREATE SEQUENCE group1_theatre_seq START WITH 1 INCREMENT BY 1 NOCACHE NOCYCLE;



CREATE SEQUENCE group1_month_seq START WITH 1 INCREMENT BY 1 NOCACHE NOCYCLE;



6.2 Client Dimension (Group1_dc)

INSERT INTO Group1_dc (client_id, client#, title_, name, town)

SELECT

group1_client_seq.NEXTVAL,

client#,

title_

name,

town

FROM (

SELECT DISTINCT

c.client#,

UPPER(TRIM(c.title)) AS title_

UPPER(TRIM(c.name)) AS name,

UPPER(TRIM(c.town)) AS town

FROM ops\$yyang00.client c

JOIN ops\$yyang00.ticketpurchase tp

ON tp.client# = c.client#

WHERE c.client# IS NOT NULL

);

6.4 Theatre Dimension (Group1_dt)

```
INSERT INTO Group1_dt (theatre_id, theatre#, name, town, country)
SELECT group1_theatre_seq.NEXTVAL,
       t.theatre#,
       UPPER(TRIM(t.name)),
       UPPER(TRIM(t.town)),
       UPPER(TRIM(t.country))
FROM ops$yyang00.theatre t
JOIN ops$yyang00.performance perf ON perf.theatre# = t.theatre#
JOIN ops$yyang00.ticketpurchase tp ON tp.per# = perf.per#;
```

6.5 Time Dimension (Group1_dm)

(corrected — removed nonexistent full_date constraint)

```
INSERT INTO Group1_dm (month_id, year, month)
SELECT group1_month_seq.NEXTVAL,
       EXTRACT(YEAR FROM perf.pdate) AS year,
       TO_CHAR(perf.pdate, 'Mon') AS month
FROM ops$yyang00.performance perf
JOIN ops$yyang00.ticketpurchase tp ON tp.per# = perf.per#
GROUP BY EXTRACT(YEAR FROM perf.pdate), TO_CHAR(perf.pdate, 'Mon');
```

6.6 Fact Table (Group1_f_ts)

(aligned with Group 2's structure but using Group 1 names)

```
INSERT INTO Group1_f_ts (prod_id, theatre_id, client_id, month_id, sale_amount)
SELECT dp.prod_id,
       dt.theatre_id,
       dc.client_id,
       dm.month_id,
       SUM(tp.totalamount)
FROM ops$yyang00.ticketpurchase tp
JOIN ops$yyang00.performance perf ON perf.per# = tp.per#
JOIN ops$yyang00.production p ON p.p# = perf.p#
JOIN ops$yyang00.theatre t ON t.theatre# = perf.theatre#
JOIN ops$yyang00.client c ON c.client# = tp.client#
JOIN Group1_dp dp ON dp.p# = p.p#
JOIN Group1_dt dt ON dt.theatre# = t.theatre#
JOIN Group1_dc dc ON dc.client# = c.client#
JOIN Group1_dm dm ON dm.year = EXTRACT(YEAR FROM perf.pdate)
                AND dm.month = TO_CHAR(perf.pdate, 'Mon')
GROUP BY dp.prod_id, dt.theatre_id, dc.client_id, dm.month_id;
```

6.7 Validation Counts

```
SELECT COUNT(*) AS client_rows FROM Group1_dc;
SELECT COUNT(*) AS prod_rows   FROM Group1_dp;
SELECT COUNT(*) AS theatre_rows FROM Group1_dt;
SELECT COUNT(*) AS time_rows   FROM Group1_dm;
SELECT COUNT(*) AS fact_rows   FROM Group1_f_ts;
```

Evidence of ETL code works

- Task 7: Based on the discussion within your group, comment on how your data mart satisfies the requirements of MT. Implement the required data analysis requirements for both the data mart and the original OLTP database. Compare your queries and comments on the advantages of a data mart in analysis operations. Test results from Oracle query should be included.

- Justification of how the data mart satisfies mart requirements queries and OLTP

- **TASK 7 – Analysis Queries and Comparison (Data Mart vs OLTP)**

- **7.1 Total Sale Value of Each Production**

- **Data Mart** Group1_f_ts

-
- SELECT dp.title, SUM(fs.amount) AS total_sale_value
- FROM group1_f_sales fs
- JOIN group1_dim_production dp ON dp.prod_id = fs.prod_id
- GROUP BY dp.title
- ORDER BY total_sale_value DESC;

- **OLTP**

- SELECT p.title, SUM(tp.totalamount) AS total_sale_value
- FROM ops\$yyang00.production p
- JOIN ops\$yyang00.performance perf ON perf.p# = p.p#
- JOIN ops\$yyang00.ticketpurchase tp ON tp.per# = perf.per#
- GROUP BY p.title
- ORDER BY total_sale_value DESC;

- **7.2 Monthly Sale Value of Each Theatre**

- **Data Mart**

- SELECT dt.name AS theatre, dd.year, dd.month_name,
- SUM(fs.amount) AS monthly_sales
- FROM group1_f_sales fs
- JOIN group1_dim_theatre dt ON dt.theatre_id = fs.theatre_id
- JOIN group1_dim_date dd ON dd.date_id = fs.date_id
- GROUP BY dt.name, dd.year, dd.month_name
- ORDER BY dt.name, dd.year, dd.month_name;

- **OLTP**

- SELECT t.name AS theatre,
- EXTRACT(YEAR FROM perf.pdate) AS year,
- TO_CHAR(perf.pdate,'Mon') AS month_name,
- SUM(tp.totalamount) AS monthly_sales
- FROM ops\$yyang00.theatre t
- JOIN ops\$yyang00.performance perf ON perf.theatre# = t.theatre#
- JOIN ops\$yyang00.ticketpurchase tp ON tp.per# = perf.per#
- GROUP BY t.name, EXTRACT(YEAR FROM perf.pdate), TO_CHAR(perf.pdate,'Mon')
- ORDER BY t.name, year, month_name;

- **7.3 Top Client (s) per Theatre (by Highest Spending)**

- **Data Mart**

- WITH spend AS (
- SELECT dt.name AS theatre, dc.name AS client, SUM(fs.amount) AS spend_amt

- FROM group1_f_sales fs
- JOIN group1_dim_theatre dt ON dt.theatre_id = fs.theatre_id
- JOIN group1_dim_client dc ON dc.client_id = fs.client_id
- GROUP BY dt.name, dc.name
-),
- ranked AS (
- SELECT theatre, client, spend_amt,
- MAX(spend_amt) OVER (PARTITION BY theatre) AS max_amt
- FROM spend
-)
- SELECT theatre, client, spend_amt
- FROM ranked
- WHERE spend_amt = max_amt
- ORDER BY theatre, client;
- **OLTP**
- WITH spend AS (
- SELECT t.name AS theatre, c.name AS client, SUM(tp.totalamount) AS spend_amt
- FROM ops\$yyang00.ticketpurchase tp
- JOIN ops\$yyang00.performance perf ON perf.per# = tp.per#
- JOIN ops\$yyang00.theatre t ON t.theatre# = perf.theatre#
- JOIN ops\$yyang00.client c ON c.client# = tp.client#
- GROUP BY t.name, c.name
-),
- ranked AS (
- SELECT theatre, client, spend_amt,
- MAX(spend_amt) OVER (PARTITION BY theatre) AS max_amt
- FROM spend
-)
- SELECT theatre, client, spend_amt
- FROM ranked
- WHERE spend_amt = max_amt
- ORDER BY theatre, client;

- **7.4 Comparison and Advantages of Data Mart Approach**

Aspect	OLTP Query	Data Mart Query
Performance	Joins multiple transactional tables (TicketPurchase + Performance + Production / Theatre / Client); heavy group by and EXTRACT()	Pre-aggregated fact table; surrogate keys and clean joins → faster response
Complexity	Harder joins, column transformations, duplicate records possible	Simplified star schema joins between dimensions and fact
Flexibility	Transaction data structure limits reporting	Clean dim/fact design enables new metrics without impacting source

Aspect	OLTP Query	Data Mart Query
Integrity & Quality	Raw data with inconsistencies and repetition	ETL cleans, trims and standardises data
Efficiency	Consumes OLTP system resources	Offloads analytical load to separate data mart

- **Conclusion:**
Both OLTP and Data Mart produce the same numerical results, but the Data Mart is superior for analytics because of cleaner joins, surrogate keys, and better query performance. This design fully meets MT's requirements for analyzing production sales, monthly theatre revenue, and top-spending clients.

-
- ☒ **Deliverable Summary**

Table	Sequence	ETL Section	Verification
group1_dim_client	group1_client_seq	6.2	COUNT(*) check
group1_dim_production	group1_prod_seq	6.3	☒
group1_dim_theatre	group1_theatre_seq	6.4	☒
group1_dim_date	group1_month_seq	6.5	☒
group1_f_sales	—	6.6	☒
Analytical Queries	—	7.1–7.3	Verified vs OLTP